

Design Rationale: Chess Position Analysis

1. The problem

My program analyses chess positions. A position comes in as a FEN string and the user gets a report about it. There are two things that change independently:

- **What kind of report** the user wants. `ScoreReport` gives a short score line. `CoachReport` gives a verdict and advice that depends on the phase of the game.
- **Which engine** evaluates the position. `PositionalEngine` looks at development and the center, so it fits the opening. `MaterialEngine` counts pieces, which is enough for the middlegame. For endgames the chess club already has an old engine, `DosChessEngine`, which knows about king activity. This is legacy code: I am not allowed to change it.

Without a pattern I would need a class for every pair: `ScoreReportWithMaterial`, `CoachReportWithDos` and so on. 2 reports $\times$ 3 engines = 6 classes, and every new report or engine adds several more.

2. The design

Bridge. `PositionReport` is the Abstraction. It has a protected final `ChessEngine` engine and knows only this interface. `ScoreReport` and `CoachReport` are Refined Abstractions. `ChessEngine` is the Implementor with `name()` and `evaluate(Position, int depth)`. It returns an `Evaluation` (score in centipawns from White's side) and reports problems only with `EngineException` and its two subclasses, `InvalidPositionException` and `EngineTimeoutException`. The implementors are `MaterialEngine`, `PositionalEngine` and `DosEngineAdapter`.

Adapter. `DosEngineAdapter` implements `ChessEngine` and wraps a `DosChessEngine` object. It converts the input, calls the legacy method, converts the result back, and translates every error code into an exception from the contract.

Complexity module: dynamic implementor selection. `PhaseRoutingEngine` is also a `ChessEngine`. It holds a `Map<GamePhase, ChessEngine>` and on every call asks the position for its phase (`Position.phase()` looks at the number of pieces and the move number) and delegates to the engine for that phase. So the engine is chosen from the input itself. The client (App) builds the map once and never decides which engine evaluates which position. Endgame positions go to the adapted legacy engine automatically.

3. Why one pattern alone is not enough

- **Bridge alone** needs every implementor to implement `ChessEngine`. `DosChessEngine` does not and I cannot edit it, so it could not be plugged into the bridge.
- **Adapter alone** would make the legacy engine usable, but the reports would still be tied to concrete engines. Every new report type would have to be written again for each engine, which is the class explosion from section 1.

Bridge separates the two hierarchies, and Adapter lets the one incompatible class join the Implementor side without touching its code.

4. Why DosChessEngine is genuinely incompatible

	ChessEngine contract	DosChessEngine
Method	<code>evaluate(Position, int depth)</code>	<code>think(int plies, char[] board, boolean blackToMove)</code>
Parameter order	position first, depth second	depth first, board second
Depth unit	full moves	plies (half-moves), so the adapter multiplies by 2
Board type	<code>Position</code> object	<code>char[64]</code> from a8 to h1, ' . ' for an empty square
Result	<code>Evaluation</code> from White's side	raw int from the side to move, so the adapter flips the sign when Black is to move
Failure	throws <code>EngineException</code> subclasses	returns the sentinel <code>NO_SCORE = -32000</code> and keeps an error code in <code>getLastError()</code>

The adapter translates the failures like this, so nothing legacy-specific reaches the reports:

Legacy error code	Exception thrown by the adapter
ERR_ILLEGAL_BOARD	InvalidPositionException ("the position is not legal")
ERR_NO_KING	InvalidPositionException (same message as the native engines)
ERR_OUT_OF_TIME	EngineTimeoutException
any other code	plain EngineException, the code is not included in the message

The adapter also fails if the error code is set when the score looks normal, so a bad result can never pass as a real score.

5. Open/Closed on both axes

- **New report:** extend `PositionReport` and implement `build()`. No engine class changes.
- **New engine:** implement `ChessEngine` and put it into the map in `App.createEngine()`. No report class and no router code changes.

`OpenClosedTest` shows both: it adds a new report and a new engine inside the test and uses them with the existing classes.

6. Limitation

The router picks exactly one engine per phase and has no fallback. If the chosen engine fails, for example the legacy engine times out at a high depth, the report fails too, even though another engine could have given an answer.